Selective Recovery for Cascading Failures in Multi-Agent LLM Systems

Lucas Yan Atharv Ramesh Utkarsh Ranjan Ethan Chen Bijin Kurian Jiby

CSE 291A Phase 1 Report (Option 5: Failure Propagation and Recovery in Multi-Agent Systems)

Question

In a multi-agent system, one agent may write incorrect code, a tester may report that it passed, and a reviewer may approve it. Restarting fixes the task but repeats work that was already correct. We ask: **given the location of a fault, can recovery re-execute only the steps that depend on it and still reach a correct outcome?** Every method below is given the fault location; automated detection is future work.

Related Work

Prior work on multi-agent failures focuses on diagnosis. Cemri et al. build a taxonomy of fourteen failure types from 1600+ traces [1]. Zhang et al. show that the best automated method identifies the responsible agent in 53.5% of runs and the decisive step in 14.2% [2]. Xie et al. model collaboration as a dependency graph through which one error can propagate [3]. UndoBench separates task competence from recovery ability, but does not measure how much of a completed run can be preserved [4]. Compensating transactions undo an action semantically rather than restoring prior state [5]. Closest to us, Wang et al. repair a memory-augmented agent after a known faulty memory: they trace dependent steps, keep steps with independent evidence, and replay the rest [6].

Experimental Setup

Travel booking (seven steps). Flight and hotel researchers query a simulated catalog, a planner combines their findings, and an executor books the flight, books the hotel, writes an itinerary, and emails the traveler. A run succeeds when exactly the correct flight and hotel are booked, the itinerary and email name both, and no email goes to another recipient.

Coding (nine steps). A specification agent writes requirements, a coder writes a function, a test tool runs it, and a reviewer records the result; a documentation step summarizes the specification; then specification, code, test, and review run again. The final function is scored on held-out tests no agent sees.

Protocol. We inject one fault into one step and let the workflow finish. Each method recovers an identical copy of that faulty run and is told the fault location. Cost is measured over the recovery pass only, in tokens (text units the model reads or writes) and seconds.

Reproducibility. The code, the 50 pinned MBPP tasks, and the run commands are at `https://github.com/UtkarshRjn/cs291a-agent-recovery`. Raw rows and traces for every Qwen number are in `artifacts/phase1/`, with the commit that produced them; `analysis/phase1_tables.py` rebuilds those tables.

Baseline: Coarse Lineage Recovery

During execution the system records, for each step, which earlier outputs appeared in its input. This record forms a dependency graph. Given a faulty output, coarse lineage recovery marks every step reachable from it in this graph, undoes the bookings and file writes those steps produced, and re-executes them. Steps with no path from the faulty output are reused unchanged.

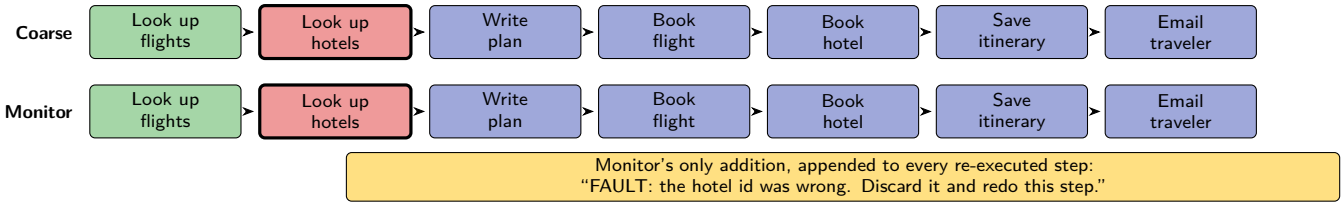


Figure 1. Recovery after a wrong hotel id. green reused; red faulty step, re-executed; blue re-executed because it read the fault. Monitor re-executes the same steps as coarse and only injects the known fault into their prompts.

We compare against two simpler policies. *Restart* undoes all effects and re-executes the entire workflow. *Suffix* re-executes the faulty step and every later step in execution order, including steps that never read the faulty output.

Travel results. We used Qwen2.5-7B-Instruct (Q4_K_M, temperature 0) on two task seeds with eleven fault types. Eight fault types changed the booking outcome, yielding 16 faulty runs.

Method	Recovered	Steps redone	Tokens	Seconds
Restart	16/16	7.0/7	6,678	17.4
Suffix	16/16	5.5/7	4,531	12.0
Coarse	16/16	4.6/7	3,650	9.9

All three policies recover every run. Coarse lineage recovery re-executes 34% fewer steps than restart and uses 45% fewer tokens and 43% less time.

Coding results. We adapted 50 problems from sanitized MBPP and evaluated 10 with GPT-OSS 120B. Of 120 scheduled fault injections (Appendix A), 96 modified a step. Restart recovered 95/96, re-executing all 9 steps at 3,340 tokens and 31.8 seconds. Coarse recovered 86/96, re-executing 5.0 steps at 1,607 tokens and 13.9 seconds. Appendix B reports results by fault placement. Coarse matches restart on ten of twelve placements and recovers one additional bad-patch case. It fails all 10 stale-report cases: the stale report is marked faulty, but the incorrect code that produced it is not, so recovery re-runs the tests and retains the code.

Proposed: Fault-Note Monitor

The monitor re-executes exactly the steps selected by coarse lineage recovery. Its only addition is a fault note: before each re-executed step, it appends the identity of the faulty output and an instruction to discard it (Figure 1). The note is built

from the fault location the system has already been given. The monitor makes no additional model call and does not perform fault detection.

Travel results. On the same 16 faulty runs, the monitor recovered 16/16 and re-executed the same 4.6 of 7 steps. Mean recovery time decreased from 9.9 to 9.6 seconds (3%), while token cost increased from 3,650 to 3,715 (2%).

Method	Recovered	Steps redone	Tokens	Seconds
Coarse	16/16	4.6/7	3,650	9.9
Monitor	16/16	4.6/7	3,715	9.6

Support rule. Following Wang et al. [6], a step is kept if it did not cite the faulty fact. We implement their citation check for one recorded link (Appendix C). On the wrong hotel id it keeps the correct flight booking that coarse cancels and rebooks (5 vs. 6 of 7 steps). Over all 16 runs it recovered 16/16 at 4.5 steps, 3,578 tokens, and 9.7 s; adding the fault note gave 3,641 tokens and 9.0 s.

Limitations. Each condition was run once. The time reduction does not hold on every task: on two short coding tasks where coarse already re-executed all nine steps, the note lengthened outputs (`clamp`: 8.2 s/1,453 tokens to 13.9 s/2,109 tokens), though tests still passed. On small synthetic graphs the note produced three incorrect outputs where coarse was correct. The GPT-OSS and Muse coding runs (Appendix B) did not include the monitor. The booking workload is simulated, and its success check omits the refundability sentence, which Qwen never writes (Appendix D).

Phase 2 Direction

The measurements point to three gaps. First, dependency tracking alone fails all 10 stale-report cases, because the faulty program sits upstream of the faulty report; recovery must reopen the step that wrote the code when re-testing fails. Second, the monitor's benefit is small and comes from single runs on one workload, so it needs repeated seeds and the MBPP faults before any claim. Third, one-hop support saves a step on the hotel fault but drops needed work on missing requirements; Wang et al.'s full sufficiency check [6] is the principled replacement.

Milestone	**Work**	**Completion check**
Phase 1 (done)	Coding and booking workloads, eleven booking and seven coding faults, dependency recording, restart, suffix, coarse, monitor, and support.	Measurements above; Qwen rows and traces in the repository.
Phase 2, week 1	Reopen the code-writing step when re-testing fails. Run monitor and support on the same ten MBPP tasks.	Stale-report cases recovered; rescue rates matched case by case on the 96 injected runs.
Phase 2, week 2	Implement Wang et al.'s sufficiency check. Test restored files, duplicate bookings, and sent emails. Compare with restart and always-verify at matched budgets.	Correctness, reused work, and cost per method on identical faulty runs.
Phase 2, week 3	All 50 MBPP tasks; at least five booking seeds and two prompt variants.	Means with confidence intervals; accidental and adversarial faults reported separately.
Final	Verify each claim against saved runs; package code, data, logs, and the report.	Every reported number rebuilt from committed rows.
Optional	A detector that must locate the fault before recovery runs.	False alarms, missed faults, and effect on recovery cost.

Table 1. Phase 2 milestones.

Appendix A: Faults Used in the Coding Evaluation

Each task is tested with one fault at a time. The first five fault types are tested separately in both coding passes; false reports target the first pass, and stale reports target the second. This gives 12 fault/stage combinations per task, or 120 attempted injections across ten tasks.

Fault	What is changed	Category	Attempts	Injected
Bad patch	Replace the function body with `return None`, in pass 1 or 2.	Accidental	20	20
Comparison change	Flip a comparison such as $>$ to $\geq$, in pass 1 or 2.	Accidental	20	10
Missing requirement	Remove a selected requirement from the specification, in pass 1 or 2.	Accidental	20	6
Tester timeout	Replace feedback with a simulated timeout, in pass 1 or 2.	Accidental	20	20
Instruction hijack	Tell the coder to ignore the task and return `None`, in pass 1 or 2.	Adversarial	20	20
False test report	Report that faulty code passed its feedback test, in pass 1.	Adversarial	10	10
Stale test report	Supply an old passing report for changed code, in pass 2.	Accidental	10	10
Total			**120**	**96**

Table 2. Scheduled and actual fault injections for GPT-OSS 120B. A comparison can only be changed if suitable code exists, and a requirement can only be removed if it appears in the specification. Ten comparison changes and fourteen requirement deletions did not apply. They are not counted as successful defenses. Each recovery method is evaluated on the same 96 runs where a fault was added.

Appendix B: GPT-OSS Recovery by Fault Placement

Restart means starting the job over. Suffix means redo the broken step and every later step. Coarse is the selective repair from the body. Each cell is how many repairs succeeded, then the average number of steps redone out of nine. A row is missing from the 96 when the planted mistake had nothing to change. Suffix and coarse match restart on ten rows. On a bad first draft they pass the run restart misses. On the stale test report both score 0/10.

Fault and stage	Restart	Suffix	Coarse
Tester timeout, pass 1	10/10 · 9.00	10/10 · 7.00	10/10 · 6.00
Tester timeout, pass 2	10/10 · 9.00	10/10 · 2.00	10/10 · 2.00
Bad patch, pass 1	9/10 · 9.00	10/10 · 8.00	10/10 · 7.00
Bad patch, pass 2	10/10 · 9.00	10/10 · 3.00	10/10 · 3.00
Missing requirement, pass 1	3/3 · 9.00	3/3 · 9.00	3/3 · 9.00
Missing requirement, pass 2	3/3 · 9.00	3/3 · 4.00	3/3 · 4.00
Instruction hijack, pass 1	10/10 · 9.00	10/10 · 9.00	10/10 · 9.00
Instruction hijack, pass 2	10/10 · 9.00	10/10 · 4.00	10/10 · 4.00
False test report, pass 1	10/10 · 9.00	10/10 · 7.00	10/10 · 6.00
Stale test report, pass 2	10/10 · 9.00	0/10 · 2.00	0/10 · 2.00
Comparison change, pass 1	5/5 · 9.00	5/5 · 8.00	5/5 · 7.00
Comparison change, pass 2	5/5 · 9.00	5/5 · 3.00	5/5 · 3.00

On the stale test report, suffix and coarse are short because they run the tests again and do not rewrite the program: 389 tokens and 2.64 seconds, and 387 tokens and 2.57 seconds. Starting over rewrites the program, uses 3,290 tokens and 33.84 seconds, and passes all 10. Both shorter repairs score 0/10.

Appendix C: Support Rule on a Wrong Hotel Id

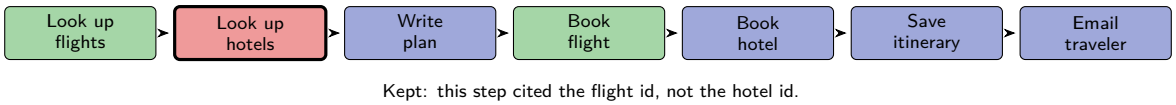


Figure 2. The support rule from Wang et al., on the same wrong hotel id. The flight booking is green here. In the coarse row of Figure 1 it was blue, so coarse cancelled a correct flight and booked it again.

Appendix D: Additional Results

Booking. Three of the eleven injected faults (a false refundability claim, a malicious hotel instruction, and an inexecutable plan) did not change the booking and were excluded. The model never stated refundability, even on clean runs, so refundability is reported separately from success. Emails already sent cannot be retracted; recovery sends a corrected email and both remain in

the record.

Muse Glimmer 30B. On the same ten MBPP problems, Muse passed 9/10 clean runs. With an injected fault, 57/95 scored runs passed without recovery; restart, suffix, and coarse recovered 87/95, 78/95, and 77/95 in 72.0, 43.2, and 40.4 seconds (4,207, 2,478, and 2,323 tokens). Four runs ended in execution errors.

Always-verify. A forward run that checks each specification, code output, and document, with one replacement allowed, passed 106/120 scheduled fault cases for both GPT-OSS and Muse. This denominator includes injections that did not apply, so it is not ranked against recovery.

Travel authorization variants. GPT-OSS passed 2/3 clean variants; the planning-only variant failed without an attack, and restart turned one correct run incorrect. Muse passed all three variants under every condition, with no unauthorized action attempted.

External benchmarks. Adapters for UndoBench (state-changing tasks), A2ASecBench (inter-agent attacks), and Who&When (failure logs) are in the repository. They do not yet recover a completed action.

References

[1] M. Cemri, M. Z. Pan, S. Yang, et al. *Why Do Multi-Agent LLM Systems Fail?* NeurIPS 2025 Datasets and Benchmarks Track. https://proceedings.neurips.cc/paper_files/paper/2025/hash/b1041e52d3be19f0a9bc491657488e4a-Abstract-Datasets_and_Benchmarks_Track.html

[2] S. Zhang, M. Yin, J. Zhang, et al. *Which Agent Causes Task Failures and When? On Automated Failure Attribution of LLM Multi-Agent Systems.* ICML 2025. arXiv:2505.00212. https://arxiv.org/abs/2505.00212

[3] Y. Xie, et al. *Modeling and Mitigating Error Cascades in LLM-Based Multi-Agent Systems.* arXiv:2603.04474, 2026. https://arxiv.org/abs/2603.04474

[4] T. Sah, et al. *UndoBench: Separating Task Competence from Recovery Capability in Tool-Using AI Agents.* arXiv:2610.05622, 2026. https://arxiv.org/abs/2610.05622

[5] H. Garcia-Molina and K. Salem. *Sagas.* In Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data, 1987. https://dl.acm.org/doi/10.1145/38714.38742

[6] Y. Wang, C. Yu, J. Zhang, Y. Duan, M. Zheng, Z. Wu, K. Shi, and T. Cai. *From Faulty Memories to Corrected Actions: Dependency-Guided Rollback Repair for Memory-Augmented Agents.* arXiv:2608.10502, 2026. https://arxiv.org/abs/2608.10502